Modern Reinforcement Learning: From Agents to LLMs

Model-Free Prediction and Control

Lecture 2

Keywords: model-free learning, Monte Carlo, temporal-difference, bootstrapping, prediction, control, on-policy, off-policy, Q-learning, SARSA

Ilija Bogunovic

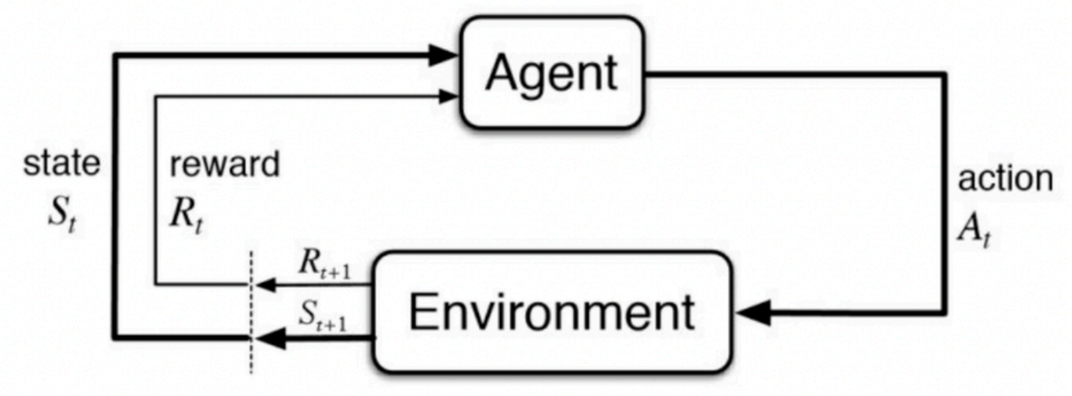
🎯 Goal

Understand how agents learn directly from experience without knowing the environment model

- We will:
  - Learn how to estimate value functions from sampled trajectories
  - Compare **Monte Carlo** (full-return) and **Temporal-Difference** (bootstrapped) methods
  - Extend prediction to **control** via **SARSA** and **Q-learning**

From Dynamic Programming to Learning from Experience

- Lecture 1: **full knowledge** of transition dynamics $P(s' \mid s, a)$ and rewards $r(s, a)$
- But in most realistic settings, the model is **unknown** or **too complex**
- Agents must **learn directly from interaction** with the environment



Why Model-Free Methods Matter

- Model-free RL ≈ **learn by doing** — estimate value from sampled experience
- No need for explicit environment equations
- Works in: games, robotics, reasoning, decision-making, dialogue systems
- Core challenge: balancing **exploration**, **variance**, and **efficiency**

$$\hat{V}(s) \approx \mathbb{E}\left[G_t \mid s_t = s\right]$$ estimated from sampled returns

Experience as Data

- Each episode or trajectory provides samples of:

$$(s_t, a_t, r_{t+1}, s_{t+1})$$

- From these, we can approximate expectations → learn V and Q
- Think of this as **data-driven value estimation**
- Learning signal comes from temporal structure of trajectories
- Rewards provide credit assignment to earlier states

The Two Fundamental Approaches

Approach	Core Idea	Pros	Cons
Monte Carlo	Learn from **complete returns** after each episode	Unbiased	High variance; episodic only
Temporal-Difference (TD)	Learn from **bootstrapped one-step targets**	Efficient, online	Biased estimates

→ Two complementary paths to learn value from experience — both crucial for scalable reasoning and decision systems

Monte Carlo: Learn from Complete Experience

- Intuition: Learning by averaging returns
- Monte Carlo (MC) methods estimate value functions **from sampled returns**
- Learn by averaging observed outcomes over many episodes

Each episode → one full return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$

Monte Carlo: Learn from Complete Experience

- Intuition: Learning by averaging returns
- Monte Carlo (MC) methods estimate value functions **from sampled returns**
- Learn by averaging observed outcomes over many episodes

Each episode → one full return $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$

Idea: the vlaue of a state is its **average return** over many visits

$$V(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_t^{(i)} \quad \text{where } s_t^{(i)} = s$$

Estimating State Values from Sampled Returns

- **Goal:** estimate value function $V(s) = \mathbb{E}[G_t \mid s_t = s]$
- We do this by **averaging observed returns** for each state:
  - Each time state s is visited, record the return G_t
  - Average all returns observed for s
- This gives an **unbiased estimate** of $V(s)$

Estimating State Values from Sampled Returns

- **Goal:** estimate value function $V(s) = \mathbb{E}[G_t \mid s_t = s]$
- We do this by **averaging observed returns** for each state:
  - Each time state s is visited, record the return G_t
  - Average all returns observed for s
- This gives an **unbiased estimate** of $V(s)$
- Works only for **episodic tasks** where full G_t can be computed

$$V(s) \leftarrow V(s) + \alpha\left[G_t - V(s)\right]$$ (incremental mean update form)

Monte Carlo Prediction Example: Learning from Full Episodes

What Happens After One Episode

- Episode: $s_1 \rightarrow s_2 \rightarrow s_3 \rightarrow s_4$(terminal)
- After the episode finishes, you compute returns: G_1 for s_1, G_2 for s_2 , G_3 for s_3
- Then: Update each visited state using its own return

$$V(s_1) \leftarrow V(s_1) + \alpha(G_1 - V(s_1))$$

$$V(s_2) \leftarrow V(s_2) + \alpha(G_2 - V(s_2))$$

$$V(s_3) \leftarrow V(s_3) + \alpha(G_3 - V(s_3))$$

- Terminal typically not updated (value fixed = 0)

Two Ways to Average Returns: First-Visit vs. Every-Visit MC

- **First-Visit MC**
  - Use only the first occurrence of each state within an episode
  - Each state is updated at most once per episode
  - Update: $V(s) \leftarrow V(s) + \alpha\left[G_t - V(s)\right]$ where t is the first time $s_t = s$
- **Every-Visit MC**
  - Use all occurrences of a state within an episode
  - Update after every visit: $V(s) \leftarrow V(s) + \alpha\left[G_t - V(s)\right]$

Convergence by the Law of Large Numbers

Monte Carlo estimate: $\hat{V}(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_i(s)$

Law of Large Numbers (LLN):

Convergence by the Law of Large Numbers

Monte Carlo estimate: $\hat{V}(s) \approx \frac{1}{N(s)} \sum_{i=1}^{N(s)} G_i(s)$

Law of Large Numbers (LLN):

- Interpretation
  - Each episode provides a noisy sample of the true value
  - Averaging across many episodes reduces noise while preserving the expectation
- Properties
  - Unbiased: expected estimate equals the true value
  - High variance: stochastic returns lead to slow convergence
  - Episodic only: requires full return G_t cannot update before the episode ends

From Prediction to Control: Monte Carlo with Exploring Starts

Goal: Learn optimal policy π^* using Monte Carlo estimates of action values

Key idea:

- Use **returns from episodes** to estimate $Q(s, a)$

- Apply **Generalized Policy Iteration (GPI)**:

From Prediction to Control: Monte Carlo with Exploring Starts

Goal: Learn optimal policy π^* using Monte Carlo estimates of action values

Key idea:

- Use **returns from episodes** to estimate $Q(s, a)$

- Apply **Generalized Policy Iteration (GPI)**:

Exploring starts assumption:

- Each episode starts with a random (s, a) pair

- Ensures all actions are tried infinitely often

Consequence:

- Monte Carlo control converges to optimal Q^* and π^*

- In practice, exploring starts are hard to guarantee → leads to ε-greedy exploration (next slide).

Balancing Exploration and Exploitation

- Motivation: Exploring starts are impractical — we need exploration within episodes

- ε-greedy policy $\pi(a \mid s)$:

$$a = \begin{cases} \arg\max_a Q(s, a) & \text{with prob. } 1 - \varepsilon \\ \text{random action} & \text{with prob. } \varepsilon \end{cases}$$

Balancing Exploration and Exploitation

- Motivation: Exploring starts are impractical — we need exploration within episodes

- ε-greedy policy $\pi(a \mid s)$:

$$a = \begin{cases} \arg\max_a Q(s,a) & \text{with prob. } 1-\varepsilon \\ \text{random action} & \text{with prob. } \varepsilon \end{cases}$$

Algorithm (on-policy MC control): Repeat:

1) Generate episodes using current ε-greedy π

2) Evaluate $Q(s,a)$ by averaging sample returns

3) Improve policy → make it greedy w.r.t. updated Q

Guarantee: With sufficient exploration $\varepsilon > 0$ and GLIE (greedy in the limit with infinite exploration),

on-policy MC control → Q^* and π^*

Off-Policy Monte Carlo Control (Importance Sampling)

- **Idea:** Learn the target policy π using episodes generated by a different behavior policy b ($\pi \neq b$)

- **Goal:** Estimate $V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t \mid S_t = s\right]$ using data collected from b

Off-Policy Monte Carlo Control (Importance Sampling)

- **Idea:** Learn the target policy π using episodes generated by a different behavior policy b ($\pi \neq b$)

- **Goal:** Estimate $V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t \mid S_t = s\right]$ using data collected from b

Off-Policy Monte Carlo Control (Importance Sampling)

- **Idea:** Learn the target policy π using episodes generated by a different behavior policy b ($\pi \neq b$)

- **Goal:** Estimate $V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_t \mid S_t = s\right]$ using data collected from b

- Importance Sampling (IS):
  - Correct returns via likelihood ratio:

$$G_t^{\text{corr}} = \rho_t G_t, \qquad \rho_t = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}$$

Interpretation:
- Corrects mismatch between behavior and target policies
- Enables reuse of experience from any data-collecting policy

Off-Policy Monte Carlo Control (Importance Sampling)

- **Idea:** Learn the target policy π using episodes generated by a different behavior policy b ($\pi \neq b$)

- **Goal:** Estimate $V^\pi(s) = \mathbb{E}_\pi\left[G_t \mid S_t = s\right]$ using data collected from b

- Importance Sampling (IS):
  - Correct returns via likelihood ratio:

$$G_t^{\text{corr}} = \rho_t G_t, \qquad \rho_t = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}$$

- Estimators:

Ordinary IS: $V^\pi(s) = \frac{1}{N} \sum_{i=1}^{N} \rho_t^{(i)} G_t^{(i)}$

Weighted IS (preferred): $V(s) \leftarrow \frac{\sum_i \rho_i G_i(s)}{\sum_i \rho_i}$ (normalize weights → lower variance)

Interpretation:
- Corrects mismatch between behavior and target policies
- Enables reuse of experience from any data-collecting policy

Summary: Monte Carlo Learning & What Comes Next

- Core idea: Learn $V^\pi(s)$ or $Q^\pi(s, a)$ from sampled episodes. Update after the episode ends:

$$V(s) \leftarrow V(s) + \alpha\,[G_t - V(s)]$$

- Variants:
  - First-Visit vs Every-Visit
  - On-Policy (ε-greedy) vs Off-Policy (Importance Sampling)
- Strengths:
  - Unbiased estimates (directly from data)
  - Simple to implement, no model required
- Weaknesses:
  - High variance → slow convergence
  - Naturally suited to episodic settings only

📘 *Temporal-Difference (TD)* updates **after every step**, combining sampling with **bootstrapping.**

Temporal-difference (TD) learning

Learning from Incomplete Episodes: Temporal-Difference Methods

- Motivation
  - Monte Carlo (MC): learns from complete returns G_t
  - Dynamic Programming (DP): bootstraps using the true value $V(s')$
  - Temporal-Difference (TD): bootstraps using the current estimate $\hat{V}(s')$

Learning from Incomplete Episodes: Temporal-Difference Methods

- Motivation
  - Monte Carlo (MC): learns from complete returns G_t
  - Dynamic Programming (DP): bootstraps using the true value $V(s')$
  - Temporal-Difference (TD): bootstraps using the current estimate $\hat{V}(s')$

- TD(0) Update Rule (updates afer every transition):

$$V(S_t) \leftarrow V(S_t) + \alpha\left[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\right]$$

- TD Error (Bellman Residual)

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

- Incremental form:

$$V(S_t) \leftarrow V(S_t) + \alpha\delta_t$$

Learning from Incomplete Episodes: Temporal-Difference Methods

Monte Carlo Target (Full Return): $$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots + \gamma^{T-t-1} R_T$$

Learning from Incomplete Episodes: Temporal-Difference Methods

Monte Carlo Target (Full Return): $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots + \gamma^{T-t-1} R_T$

TD(0) Target (One-Step Return): $\text{Target}_t^{\text{TD}} = R_{t+1} + \gamma V(S_{t+1})$

Learning from Incomplete Episodes: Temporal-Difference Methods

Monte Carlo Target (Full Return): $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots + \gamma^{T-t-1} R_T$

TD(0) Target (One-Step Return): $\text{Target}_t^{\text{TD}} = R_{t+1} + \gamma V(S_{t+1})$

TD(0) Update: $V(S_t) \leftarrow V(S_t) + \alpha\left(\text{Target}_t^{\text{TD}} - V(S_t)\right)$

Learning from Incomplete Episodes: Temporal-Difference Methods

Monte Carlo Target (Full Return): $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots + \gamma^{T-t-1} R_T$

TD(0) Target (One-Step Return): $\text{Target}_t^{\text{TD}} = R_{t+1} + \gamma V(S_{t+1})$

TD(0) Update: $V(S_t) \leftarrow V(S_t) + \alpha\left(\text{Target}_t^{\text{TD}} - V(S_t)\right)$

Key Differences:

- Monte Carlo (MC) uses the complete future return
- Temporal-Difference (TD) uses a bootstrapped estimate of the future return
- TD learns online, while MC must wait until the episode ends
- TD introduces bias but reduces variance → faster learning in practice

🧮 TD(0) Prediction Algorithm (for a given policy π)

‹/› Code

```
Input:  policy π, step-size α, discount γ
Initialize V(s) arbitrarily for all states s

Loop over episodes:
    Initialize S
    Repeat (for each step of episode):
        Take action A ~ π(· | S)
        Observe reward R and next state S'
        δ ← R + γ V(S') − V(S)      (TD error)
        V(S) ← V(S) + α δ           (TD update)
        S ← S'
    until S is terminal
```

🧮 TD(0) Prediction Algorithm (for a given policy π)

‹/› Code

```
Input:  policy π, step-size α, discount γ
Initialize V(s) arbitrarily for all states s

Loop over episodes:
    Initialize S
    Repeat (for each step of episode):
        Take action A ~ π(· | S)
        Observe reward R and next state S'
        δ ← R + γ V(S') − V(S)      (TD error)
        V(S) ← V(S) + α δ           (TD update)
        S ← S'
    until S is terminal
```

Notes:

- Online & incremental: updates after every transition

- Model-free: does not require transition probabilities

- Convergence: in the tabular setting with fixed policy π, converges to V^π under standard step-size conditions

TD as Stochastic Approximation to the Bellman Operator

- Bellman Expectation Equation

$$V^\pi(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s\right]$$

- Bellman Operator

$$(T^\pi V)(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s\right]$$

- Fixed-Point Property

$$V^\pi = T^\pi V^\pi$$

TD as Stochastic Approximation to the Bellman Operator

- Bellman Expectation Equation

$$V^\pi(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s\right]$$

- Bellman Operator

$$(T^\pi V)(s) = \mathbb{E}_\pi\left[R_{t+1} + \gamma V(S_{t+1}) \mid S_t = s\right]$$

- Fixed-Point Property

$$V^\pi = T^\pi V^\pi$$

- TD(0) as Stochastic Approximation

$$V(S_t) \leftarrow V(S_t) + \alpha\Big(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\Big)$$

- The term inside parentheses is a sample of

$$(T^\pi V)(S_t) - V(S_t)$$

- Key Interpretation: TD performs noisy stochastic-gradient-like steps toward the fixed point of T^π

Why TD Converges: T^π is a γ-Contraction

- γ-Contraction Property: For any value functions

$$\|T^\pi V - T^\pi U\|_\infty \leq \gamma \|V - U\|_\infty \text{ with } \quad 0 \leq \gamma < 1$$

- Consequences
  - Unique fixed point: $V^\pi = T^\pi V^\pi$
  - Iterative convergence: $V_{k+1} = T^\pi V_k \;\Rightarrow\; V_k \to V^\pi$

📉 Why TD Converges: T^π is a γ-Contraction

- γ-Contraction Property: For any value functions

$$\|T^\pi V - T^\pi U\|_\infty \leq \gamma \|V - U\|_\infty \text{ with } \quad 0 \leq \gamma < 1$$

- Consequences
  - Unique fixed point: $V^\pi = T^\pi V^\pi$
  - Iterative convergence: $V_{k+1} = T^\pi V_k \;\Rightarrow\; V_k \to V^\pi$

- TD(0): stochastic approximation of this contraction → convergence (under standard conditions)

- Key Takeaway: **contraction ⇒ unique fixed point ⇒ convergence**

Why TD(0) Converges: Stochastic Approximation View

- TD(0) Update: $V(S_t) \leftarrow V(S_t) + \alpha_t\big(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\big)$

Why TD(0) Converges: Stochastic Approximation View

- TD(0) Update: $V(S_t) \leftarrow V(S_t) + \alpha_t\big(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\big)$
- Rewrite as Noisy Fixed-Point Iteration

$$V(S_t) \leftarrow V(S_t) + \alpha_t\big((T^\pi V)(S_t) - V(S_t) + \text{noise}_t\big)$$

- where

$$\text{noise}_t = \underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{sample}} - \underbrace{\mathbb{E}_\pi[R_{t+1} + \gamma V(S_{t+1}) \mid S_t]}_{(T^\pi V)(S_t)} \quad \text{and} \quad \mathbb{E}[\text{noise}_t \mid S_t] = 0.$$

Why TD(0) Converges: Stochastic Approximation View

- TD(0) Update: $V(S_t) \leftarrow V(S_t) + \alpha_t\big(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\big)$
- Rewrite as Noisy Fixed-Point Iteration

$$V(S_t) \leftarrow V(S_t) + \alpha_t\big((T^\pi V)(S_t) - V(S_t) + \text{noise}_t\big)$$

- where

$$\text{noise}_t = \underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{sample}} - \underbrace{\mathbb{E}_\pi[R_{t+1} + \gamma V(S_{t+1}) \mid S_t]}_{(T^\pi V)(S_t)} \quad \text{and} \quad \mathbb{E}[\text{noise}_t \mid S_t] = 0.$$

- Convergence (Tabular Case)

$\sum_t \alpha_t = \infty, \quad \sum_t \alpha_t^2 < \infty,$ and all states are visited infinitely often, $V_t \to V^\pi$ with probability 1

Why TD(0) Converges: Stochastic Approximation View

- TD(0) Update: $V(S_t) \leftarrow V(S_t) + \alpha_t\big(R_{t+1} + \gamma V(S_{t+1}) - V(S_t)\big)$
- Rewrite as Noisy Fixed-Point Iteration

$$V(S_t) \leftarrow V(S_t) + \alpha_t\big((T^\pi V)(S_t) - V(S_t) + \text{noise}_t\big)$$

- where

$$\text{noise}_t = \underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{sample}} - \underbrace{\mathbb{E}_\pi[R_{t+1} + \gamma V(S_{t+1}) \mid S_t]}_{(T^\pi V)(S_t)} \quad \text{and} \quad \mathbb{E}[\text{noise}_t \mid S_t] = 0.$$

- Convergence (Tabular Case)

$\sum_t \alpha_t = \infty, \qquad \sum_t \alpha_t^2 < \infty,$ and all states are visited infinitely often, $V_t \to V^\pi$ with probability 1

- **Key Intuition:** TD = noisy iteration of a γ-contraction → noise vanishes → convergence

Example: How TD Propagates Value Backwards Step-by-Step

Example: How TD Propagates Value Backwards Step-by-Step

TD gradually pushes value backward at each timestep — not at episode end.

4 × 4 GRIDWORLD

0,0 ★ – V(0,0)	0,1 0.00 V(0,1)	0,2 0.00 V(0,2)	0,3 0.00 V(0,3)
1,0 0.00 V(1,0)	1,1 0.00 V(1,1)	1,2 0.10 V(1,2)	1,3 0.00 V(1,3)
2,0 0.00 V(2,0)	2,1 0.00 V(2,1)	2,2 0.00 V(2,2)	2,3 0.00 V(2,3)
3,0 0.00 V(3,0)	3,1 0.00 V(3,1)	3,2 0.00 V(3,2)	3,3 0.00 V(3,3)

Terminal (+1) · Agent path · Updating · Value changed

SCENARIO

Terminal state = (0,0) top-left, reward +1

All other transitions give reward 0

Agent follows a random policy, $\alpha = 0.1$, $\gamma = 1$

TD(0) UPDATES

TRANSITION 1

$S_t = (2,3) \rightarrow S_{t+1} = (2,2)$, $R = 0$

$V(2,3) \leftarrow 0 + 0.1\,(0 + V(2,2) - 0) = 0$

TRANSITION 2

$S_t = (2,2) \rightarrow S_{t+1} = (1,2)$, $R = 0$

$V(2,2) \leftarrow 0 + 0.1\,(0 + V(1,2) - 0) = 0$

TRANSITION 3

$S_t = (1,2) \rightarrow$ terminal $(0,0)$, $R = +1$

$V(1,2) \leftarrow 0 + 0.1\,(1 + V(\text{term}) - 0) = 0.1$

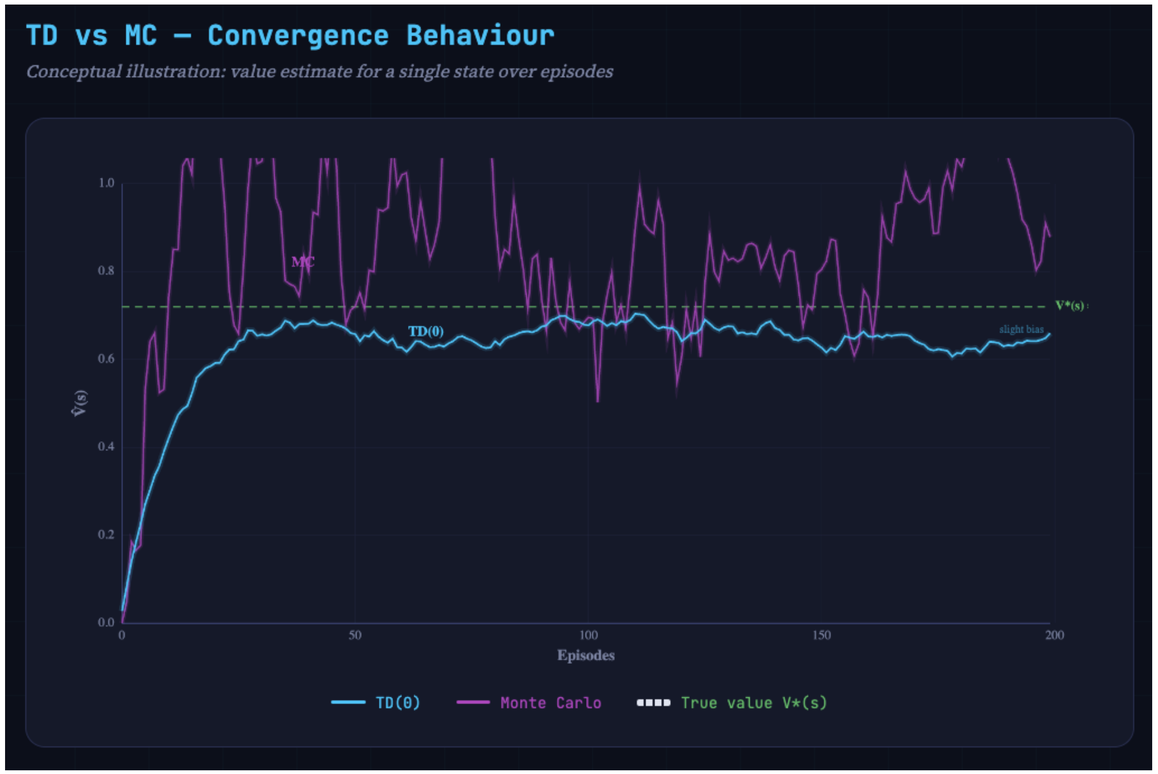
← Back | Next → | Reset

KEY INSIGHT

The +1 reward signal is **propagated backward one step at a time**. With enough episodes, TD gradually diffuses the terminal reward across the entire state space — learning *incrementally* without waiting for episodes to finish.

⚖️ TD Learns Faster, but TD Estimates Can Be Biased

- TD(0) Converges Faster
  - Updates happen online
  - Lower variance → smoother learning
  - Typically reaches good performance sooner
- ⚠️ But TD Can Be Biased (Early On)
  - Bias comes from bootstrapping TD target
  - If $V(S_{t+1})$ is wrong, the error propagates backward



- Monte Carlo
  - Produces unbiased estimates
  - But: Must wait for full episodes; High variance → noisy learning curves

📊 Monte Carlo vs. Temporal-Difference (TD)

Aspect	Monte Carlo (MC)	Temporal-Difference (TD)
Learning target	Full return G_t	One-step bootstrap $R_{t+1} + \gamma V(S_{t+1})$
Update timing	End of episode only	Every time-step
Requires episodes?	Yes (episodic only)	No (works in continuing tasks)
Bias	Unbiased	Biased (uses estimated future value)
Variance	High variance	Low variance
Data efficiency	Inefficient (delayed updates)	Efficient (online, incremental)
Dependence on model	Model-free	Model-free
Convergence	Slow	Faster, more stable

Model-free control

Model-Free Control

- What is Control in RL?
- **Goal:** learn an optimal policy π^* without knowing the model
- Only interact with the environment: observe (s, a, r, s')
- Learn optimal action–value function Q*(s, a)
- Derive policy by acting greedily with respect to Q*(s, a)

On-Policy vs Off-Policy Control

- **Two fundamental approaches:**
  - On-policy control: learn the value of the policy that is actually used to generate behavior
  - Off-policy control: learn the value of a different (usually greedy) target policy while following a more exploratory behavior policy
  - Key distinction lies in the target used during value updates

- Definitions:
  - Behavior policy: $\pi_b(a \mid s)$
  - Target policy: $\pi_t(a \mid s)$

SARSA: On-Policy TD Control (Intuition)

- **Key idea:**
  - Learn Q(s, a) while following the same policy that is being evaluated
  - Update target uses the action actually taken:

$$\text{Target} = r + \gamma Q(s', a')$$

- **Why SARSA?**
  - Naturally incorporates exploratory actions
  - Tracks the true return of the behavior policy
  - Safer in stochastic environments

- **Name:** S → A → R → S → A (state, action, reward, next state, next action)

SARSA Update Rule (On-Policy)

- TD target (on-policy):

- SARSA update:

- Characteristics:
  - On-policy: depends on the action a' selected by the current behavior policy
  - Learns Q^π where π is typically ε-greedy w.r.t. Q
  - Converges under GLIE conditions

SARSA Pseudocode (On-Policy TD Control)

- **SARSA (for action-value control):**

```
Initialize Q(s, a) arbitrarily

For each episode:
    Initialize state s
    Choose action a ~ ε-greedy(Q)

    Repeat:
        Take action a
        Observe reward r and next state s'
        Choose next action a' ~ ε-greedy(Q)

        Q(s, a) ← Q(s, a) + α [ r + γ Q(s', a') − Q(s, a) ]

        s ← s'
        a ← a'
    until s is terminal
```

Q-Learning: Off-Policy TD Control (Intuition)

- **Key idea:**
  - Learn the value of the greedy policy, even while following an exploratory behavior policy
  - Update target uses the best possible next action, not the action actually taken

- **Target:**

$$r + \gamma \max_{a'} Q(s', a')$$

- **Why Q-learning?**
  - Off-policy: behaves with ε-greedy policy, but learns about the greedy policy
  - Converges to optimal Q* under standard assumptions
  - Often learns faster toward optimal behavior

Q-Learning Update Rule (Off-Policy)

- **TD target (off-policy):**

- **Q-learning update:**

- Characteristics:
  - Off-policy: update uses greedy action, not the action actually taken
  - Learns the optimal action-value function Q*
  - Behavior policy can still be ε-greedy for exploration

Q-Learning Pseudocode (Off-Policy TD Control)

- **Q-learning algorithm:**

```
Initialize Q(s, a) arbitrarily

For each episode:
    Initialize state s

    Repeat:
        Choose action a ~ ε-greedy(Q)
        Take action a
        Observe reward r and next state s'

        Q(s, a) ← Q(s, a) + α [ r + γ max_{a'} Q(s', a') - Q(s, a) ]

        s ← s'
    until s is terminal
```

SARSA vs. Q-Learning: Summary

- SARSA (on-policy):
  - Target uses the action actually taken: Target = $r + \gamma Q(s', a')$
  - Learns Q^{π} for the ε-greedy behavior policy
  - Accounts for exploratory actions

SARSA vs. Q-Learning: Summary

- SARSA (on-policy):
  - Target uses the action actually taken: Target = $r + \gamma Q(s', a')$
  - Learns Q^π for the ε-greedy behavior policy
  - Accounts for exploratory actions

- Q-Learning (off-policy):
  - Target uses the greedy action: Target = $r + \gamma \max_{a'} Q(s', a')$
  - Learns optimal Q^* even while exploring
  - More aggressive; can converge faster

- Core difference:

SARSA evaluates the behavior policy.
Q-learning evaluates the greedy target policy.

Extensions of Q-Learning

Double Q-Learning

- Addresses Q-learning's overestimation bias
- Maintain two value tables Q_1 and Q_2
- Update with cross-estimation: Use Q_1 to select argmax, Q_2 to evaluate it (or vice versa)
- More stable and accurate in stochastic environments

Extensions of Q-Learning

Double Q-Learning

- Addresses Q-learning's overestimation bias
- Maintain two value tables Q_1 and Q_2
- Update with cross-estimation: Use Q_1 to select argmax, Q_2 to evaluate it (or vice versa)
- More stable and accurate in stochastic environments

Dyna-Q

- Integrates model learning + planning + real experience
- Learn a model from transitions $(s, a) \rightarrow (r, s')$
- Perform additional "imagined" Q-learning updates from the model
- Improves sample efficiency dramatically

Extensions of Q-Learning

Double Q-Learning

- Addresses Q-learning's overestimation bias
- Maintain two value tables Q_1 and Q_2
- Update with cross-estimation: Use Q_1 to select argmax, Q_2 to evaluate it (or vice versa)
- More stable and accurate in stochastic environments

Dyna-Q

- Integrates model learning + planning + real experience
- Learn a model from transitions (s, a) → (r, s')
- Perform additional "imagined" Q-learning updates from the model
- Improves sample efficiency dramatically

Replay Buffers (Experience Replay)

- Store past transitions in a buffer
- Sample mini-batches for off-policy Q-learning updates
- Breaks correlation between sequential samples
- Enables stable deep Q-learning (DQN)

Big Picture Summary

- **Monte Carlo (MC) vs Temporal-Difference (TD)**
  - MC: full returns, unbiased but high variance
  - TD: sampled + bootstrapped target, biased but low variance
  - TD updates online; MC needs complete episodes
- **Prediction vs Control**
  - Prediction: learn V^π or Q^π for a fixed policy
  - Control: improve policy toward π^* using SARSA or Q-learning
  - Control requires exploration (ε-greedy)
- **Core insight**
  - TD enables scalable, online RL and forms foundation for modern algorithms

Why This Matters for LLMs

- Learning from sampled feedback
  - LLMs learn from sampled completions and reward model or preference scores
  - Bootstrapped reasoning
  - Reward model predictions serve as “value estimates”
- Updates propagate feedback backward through tokens
- Advantage estimation foundations
  - TD structure underpins advantage estimation (GAE) used in PPO/GRPO
  - Modern LLM RL fine-tuning builds directly on TD concepts

Historical Note

- **Early milestones**
  - TD learning introduced in the 1980s (Sutton)
  - SARSA, Q-learning (Watkins) define classic RL control
  - Enabled first successful RL systems in robotics, games
- **Modern impact**
  - Foundations for deep RL (DQN, Atari breakthrough in 2015)
  - Basis for PPO, GRPO, A2C/A3C advantage-based algorithms
  - Still central in modern agentic LLM training
- These “classical” algorithms remain deeply embedded in today’s large models

Reading Guide

- **Primary text:**
  - Sutton & Barto, Reinforcement Learning: An Introduction
    - Chapter 5 — Monte Carlo methods
    - Chapter 6 — Temporal-Difference learning

- **Optional deeper resources:**
  - Watkins (1989), Q-learning
  - Sutton (1988), Learning to predict by the methods of temporal differences
  - van Hasselt (2010), Double Q-learning
  - Dyna-Q and early model-based RL papers (Sutton 1990)

- Recommendation: Focus on the Bellman operator view and bias–variance intuition